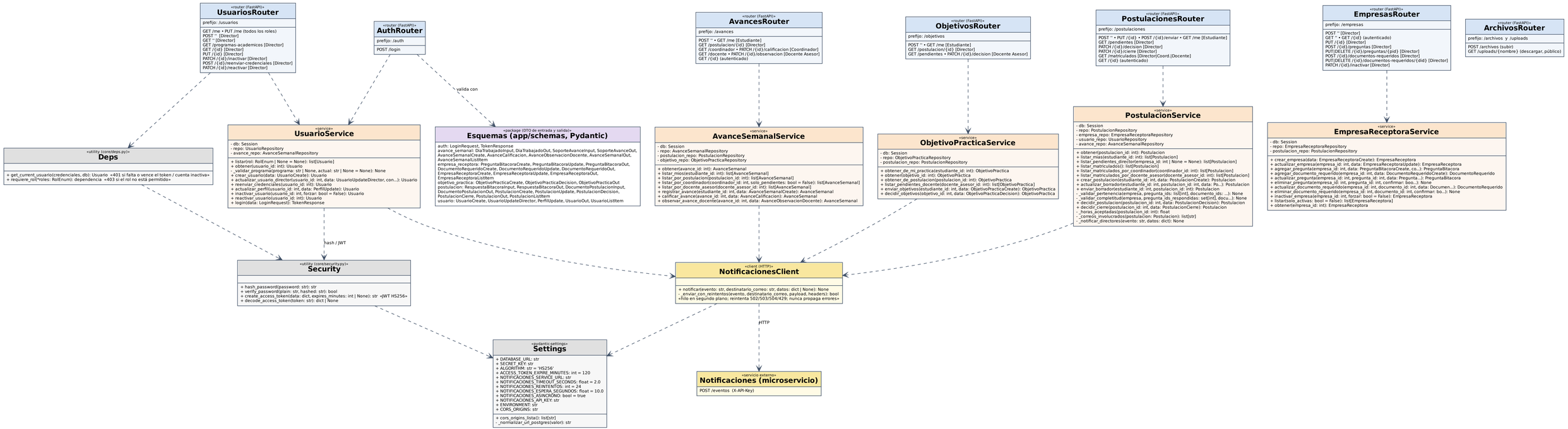
JEDANIX — Diagrama de clases (2/5): Núcleo — Routers, Servicios y componentes transversales
«router (FastAPI)»
UsuariosRouter
prefijo: /usuarios
GET /me • PUT /me (todos los roles)
POST "" [Director]
GET "" [Director]
GET /programas-academicos [Director]
GET /{id} [Director]
PUT /{id} [Director]
PATCH /{id}/inactivar [Director]
POST /{id}/reenviar-credenciales [Director]
PATCH /{id}/reactivar [Director]
«router (FastAPI)»
AuthRouter
prefijo: /auth
POST /login
valida con
«router (FastAPI)»
AvancesRouter
prefijo: /avances
POST "" • GET /me [Estudiante]
GET /postulacion/{id} [Director]
GET /coordinador • PATCH /{id}/calificacion [Coordinador]
GET /docente • PATCH /{id}/observacion [Docente Asesor]
GET /{id} (autenticado)
«router (FastAPI)»
ObjetivosRouter
prefijo: /objetivos
POST "" • GET /me [Estudiante]
GET /postulacion/{id} [Director]
GET /pendientes • PATCH /{id}/decision [Docente Asesor]
«router (FastAPI)»
PostulacionesRouter
prefijo: /postulaciones
POST "" • PUT /{id} • POST /{id}/enviar • GET /me [Estudiante]
GET /pendientes [Director]
PATCH /{id}/decision [Director]
PATCH /{id}/cierre [Director]
GET /matriculados [Director|Coord.|Docente]
GET /{id} (autenticado)
«router (FastAPI)»
EmpresasRouter
prefijo: /empresas
POST "" [Director]
GET "" • GET /{id} (autenticado)
PUT /{id} [Director]
POST /{id}/preguntas [Director]
PUT|DELETE /{id}/preguntas/{pid} [Director]
POST /{id}/documentos-requeridos [Director]
PUT|DELETE /{id}/documentos-requeridos/{did} [Director]
PATCH /{id}/inactivar [Director]
«router (FastAPI)»
ArchivosRouter
prefijo: /archivos y /uploads
POST /archivos (subir)
GET /uploads/{nombre} (descargar, público)
«utility (core/deps.py)»
Deps
+ get_current_usuario(credenciales, db): Usuario «401 si falta o vence el token / cuenta inactiva»
+ requiere_rol(*roles: RolEnum): dependencia «403 si el rol no está permitido»
«service»
UsuarioService
- db: Session
- repo: UsuarioRepository
- avance_repo: AvanceSemanalRepository
+ listar(rol: RolEnum | None = None): list[Usuario]
+ obtener(usuario_id: int): Usuario
- _validar_programa(programa: str | None, actual: str | None = None): None
+ crear_usuario(data: UsuarioCreate): Usuario
+ actualizar_usuario_director(usuario_id: int, data: UsuarioUpdateDirector, con...): Usuario
+ reenviar_credenciales(usuario_id: int): Usuario
+ actualizar_perfil(usuario_id: int, data: PerfilUpdate): Usuario
+ inactivar_usuario(usuario_id: int, forzar: bool = False): Usuario
+ reactivar_usuario(usuario_id: int): Usuario
+ login(data: LoginRequest): TokenResponse
«package (DTO de entrada y salida)»
Esquemas (app/schemas, Pydantic)
auth: LoginRequest, TokenResponse
avance_semanal: DiaTrabajadoInput, DiaTrabajadoOut, SoporteAvanceInput, SoporteAvanceOut, AvanceSemanalCreate, AvanceCalificacion, AvanceObservacionDocente, AvanceSemanalOut, AvanceSemanalListItem
empresa_receptora: PreguntaBitacoraCreate, PreguntaBitacoraUpdate, PreguntaBitacoraOut, DocumentoRequeridoCreate, DocumentoRequeridoUpdate, DocumentoRequeridoOut, EmpresaReceptoraCreate, EmpresaReceptoraUpdate, EmpresaReceptoraOut, EmpresaReceptoraListItem
objetivo_practica: ObjetivoPracticaCreate, ObjetivoPracticaDecision, ObjetivoPracticaOut
postulacion: RespuestaBitacoraInput, RespuestaBitacoraOut, DocumentoPostulacionInput, DocumentoPostulacionOut, PostulacionCreate, PostulacionUpdate, PostulacionDecision, PostulacionCierre, PostulacionOut, PostulacionListItem
usuario: UsuarioCreate, UsuarioUpdateDirector, PerfilUpdate, UsuarioOut, UsuarioListItem
«service»
AvanceSemanalService
- db: Session
- repo: AvanceSemanalRepository
- postulacion_repo: PostulacionRepository
- objetivo_repo: ObjetivoPracticaRepository
+ obtener(avance_id: int): AvanceSemanal
+ listar_mios(estudiante_id: int): list[AvanceSemanal]
+ listar_por_postulacion(postulacion_id: int): list[AvanceSemanal]
+ listar_por_coordinador(coordinador_id: int, solo_pendientes: bool = false): list[AvanceSemanal]
+ listar_por_docente_asesor(docente_asesor_id: int): list[AvanceSemanal]
+ registrar_avance(estudiante_id: int, data: AvanceSemanalCreate): AvanceSemanal
+ calificar_avance(avance_id: int, data: AvanceCalificacion): AvanceSemanal
+ observar_avance_docente(avance_id: int, data: AvanceObservacionDocente): AvanceSemanal
«service»
ObjetivoPracticaService
- db: Session
- repo: ObjetivoPracticaRepository
- postulacion_repo: PostulacionRepository
+ obtener_de_mi_practica(estudiante_id: int): ObjetivoPractica
+ obtener(objetivo_id: int): ObjetivoPractica
+ obtener_de_postulacion(postulacion_id: int): ObjetivoPractica
+ listar_pendientes_docente(docente_asesor_id: int): list[ObjetivoPractica]
+ enviar_objetivos(estudiante_id: int, data: ObjetivoPracticaCreate): ObjetivoPractica
+ decidir_objetivos(objetivo_id: int, data: ObjetivoPracticaDecision): ObjetivoPractica
«service»
PostulacionService
- db: Session
- repo: PostulacionRepository
- empresa_repo: EmpresaReceptoraRepository
- usuario_repo: UsuarioRepository
- avance_repo: AvanceSemanalRepository
+ obtener(postulacion_id: int): Postulacion
+ listar_mias(estudiante_id: int): list[Postulacion]
+ listar_pendientes_director(empresa_id: int | None = None): list[Postulacion]
+ listar_matriculados(): list[Postulacion]
+ listar_matriculados_por_coordinador(coordinador_id: int): list[Postulacion]
+ listar_matriculados_por_docente_asesor(docente_asesor_id: int): list[Postulacion]
+ crear_postulacion(estudiante_id: int, data: PostulacionCreate): Postulacion
+ actualizar_borrador(estudiante_id: int, postulacion_id: int, data: Po...): Postulacion
+ enviar_borrador(estudiante_id: int, postulacion_id: int): Postulacion
- _validar_pertenencia(empresa, pregunta_ids: list[int], documento_ids: ...): None
- _validar_completitud(empresa, pregunta_ids_respondidas: set[int], docu...): None
+ decidir_postulacion(postulacion_id: int, data: PostulacionDecision): Postulacion
+ decidir_cierre(postulacion_id: int, data: PostulacionCierre): Postulacion
- _horas_aceptadas(postulacion_id: int): float
- _correos_involucrados(postulacion: Postulacion): list[str]
- _notificar_directores(evento: str, datos: dict): None
«service»
EmpresaReceptoraService
- db: Session
- repo: EmpresaReceptoraRepository
- postulacion_repo: PostulacionRepository
+ crear_empresa(data: EmpresaReceptoraCreate): EmpresaReceptora
+ actualizar_empresa(empresa_id: int, data: EmpresaReceptoraUpdate): EmpresaReceptora
+ agregar_pregunta(empresa_id: int, data: PreguntaBitacoraCreate, co...): PreguntaBitacora
+ agregar_documento_requerido(empresa_id: int, data: DocumentoRequeridoCreate): DocumentoRequerido
+ actualizar_pregunta(empresa_id: int, pregunta_id: int, data: Pregunta...): PreguntaBitacora
+ eliminar_pregunta(empresa_id: int, pregunta_id: int, confirmar: boo...): None
+ actualizar_documento_requerido(empresa_id: int, documento_id: int, data: Documen...): DocumentoRequerido
+ eliminar_documento_requerido(empresa_id: int, documento_id: int, confirmar: bo...): None
+ inactivar_empresa(empresa_id: int, forzar: bool = false): EmpresaReceptora
+ listar(solo_activas: bool = false): list[EmpresaReceptora]
+ obtener(empresa_id: int): EmpresaReceptora
hash / JWT
«utility (core/security.py)»
Security
+ hash_password(password: str): str
+ verify_password(plain: str, hashed: str): bool
+ create_access_token(data: dict, expires_minutes: int | None): str «JWT HS256»
+ decode_access_token(token: str): dict | None
«client (HTTP)»
NotificacionesClient
+ notificar(evento: str, destinatario_correo: str, datos: dict) None): None
- _enviar_con_reintentos(evento, destinatario_correo, payload, headers): bool
«hilo en segundo plano; reintenta 502/503/504/429; nunca propaga errores»
HTTP
«pydantic-settings»
Settings
+ DATABASE_URL: str
+ SECRET_KEY: str
+ ALGORITHM: str = 'HS256'
+ ACCESS_TOKEN_EXPIRE_MINUTES: int = 120
+ NOTIFICACIONES_SERVICE_URL: str
+ NOTIFICACIONES_TIMEOUT_SECONDS: float = 2.0
+ NOTIFICACIONES_REINTENTOS: int = 24
+ NOTIFICACIONES_ESPERA_SEGUNDOS: float = 10.0
+ NOTIFICACIONES_ASINCRÓNO: bool = true
+ NOTIFICACIONES_API_KEY: str
+ ENVIRONMENT: str
+ CORS_ORIGINS: str
+ cors_origins_lista(): list[str]
- _normalizar_url_postgres(valor): str
«servicio externo»
Notificaciones (microservicio)
POST /eventos (X-API-Key)